```sql
-- 4.1.1 Calculate the count, total, average, minimum, and maximum of your theme's metric across the whole event table, and how many distinct actors appear.

SELECT  338 ms
    COUNT(*) AS total_trips,
    SUM(fare_amount) AS total_fare,
    AVG(fare_amount) AS average_fare,
    MIN(fare_amount) AS minimum_fare,
    MAX(fare_amount) AS maximum_fare,
    COUNT(DISTINCT rider_id) AS unique_riders
FROM trips;
```

s > Database > postgres@localhost > q4_1.sql [session: q4_1.aql]

Output | Result 9

total_trips	total_fare	average_fare	minimum_fare	maximum_fare	unique_riders
200	3628.19	18.14095	0	70.86	30

```sql
-- 4.1.2 calculate the event (trip) count and average metric for each producer (driver). Sort the results from the producer with the most events to the producer with the fewest.

SELECT  355 ms
    driver_id,
    COUNT(*) AS total_trips,
    AVG(fare_amount) AS average_fare
FROM trips
GROUP BY driver_id
ORDER BY total_trips DESC;
```

Database > postgres@localhost > q4_1.sql [session: q4_1.aql]

Output | Result 10

driver_id	total_trips	average_fare
6	21	17.8371428571428571
8	20	22.628
10	19	20.9668421052631579
5	18	23.0572222222222222
7	18	18.33
12	17	18.2711764705882353
2	16	17.021875
11	16	11.94125
9	14	14.1714285714285714
3	13	13.3569230769230769
13	11	19.8581818181818182
1	11	16.0263636363636364
4	6	19.4866666666666667

```sql
-- 4.1.3 Which producers have enough volume for their rate to mean anything? A producer with only a few events may have a high failure rate that is not representative of their usual performance.
-- Choose a minimum event-count threshold and use HAVING to include only producers who meet that threshold. Explain why you selected your threshold.

SELECT  339 ms
    driver_id,
    COUNT(*) AS total_trips,
    AVG(fare_amount) AS average_fare
FROM trips
GROUP BY driver_id
HAVING COUNT(*) >= 10
ORDER BY total_trips DESC;
```

s > Database > postgres@localhost > q4_1.sql [session: q4_1.aql]

Output | Result 12

CSV

driver_id	total_trips	average_fare
6	21	17.8371428571428571
8	20	22.628
10	19	20.9668421052631579
5	18	23.0572222222222222
7	18	18.33
12	17	18.2711764705882353
11	16	11.94125
2	16	17.021875
9	14	14.1714285714285714
3	13	13.3569230769230769
13	11	19.8581818181818182
1	11	16.0263636363636364

```sql
-- 4.1.4 How does the picture change along two dimensions at once? Leadership also wants to understand how performance varies across regions.
-- Write a query that groups the data by two dimensions, including region, and calculates the relevant summary statistics.
-- Your results should contain one row for each combination of the two dimensions so that regional leaders can review their results.

SELECT  360 ms
    driver_id,
    pickup_city,
    COUNT(*) AS total_trips,
    AVG(fare_amount) AS average_fare
FROM trips
GROUP BY driver_id, pickup_city
ORDER BY driver_id, pickup_city;
```

Database > postgres@localhost > q4_1.sql [session: q4_1.aql]

Output | Result 13

driver_id	pickup_city	total_trips	average_fare
1	Danang	1	7.4
1	Haiphong	4	24.2175
1	Hanoi	1	7.07
1	Hue	3	6.6866666666666667
1	Saigon	2	22.445
2	Danang	2	14.36
2	Haiphong	4	18.21
2	Hanoi	5	16.806
2	Hue	2	14.71
2	Saigon	3	19.1133333333333333
3	Danang	5	17.356
3	Hanoi	3	10.9166666666666667
3	Hue	3	4.4133333333333333
3	Saigon	2	20.435
4	Danang	2	11.385
4	Haiphong	1	0
4	Hanoi	1	14.34
4	Hue	1	6
4	Saigon	1	8.95

62 rows

```sql
-- Part A: Two numbers in one pass
-- For each producer, compute the total number of events and the total number of events classified as failures, in a single pass over the event table, without a WHERE clause filtering the whole query. The outl
-- ✓  CASE used inside an aggregate — for example SUM(CASE WHEN ... THEN 1 ELSE 0 END) or COUNT(CASE WHEN ... END).
-- ✓  Both totals appearing as separate columns in one result set.
SELECT  342 ms
    driver_id,
    COUNT(*) AS total_trips,
    SUM(
        CASE
            WHEN cancellation_reason = 'Rider no show' THEN 1
            ELSE 0
        END
    ) AS rider_no_shows
FROM trips
GROUP BY driver_id
ORDER BY rider_no_shows DESC;
```

Database > postgres@localhost > q4_2.sql

Output | Result 13

CSV

driver_id	total_trips	rider_no_shows
2	16	3
3	13	2
11	16	1
6	21	1
12	17	1
9	14	1
7	18	1
1	11	1
4	6	1
13	11	0
10	19	0
8	20	0
5	18	0

```sql
-- Part B: Subtotals and a grand total
-- The regional director wants subtotals per region and a platform grand total, not the detail rows. Write a query that produces regional subtotals and a platform-wide grand total without including the ind

-- ✓  GROUP BY ROLLUP (...), or GROUPING SETS or CUBE.

-- ✓  A written note identifying which rows are the subtotals and which is the grand total.

SELECT  341 ms
    CASE
        WHEN GROUPING(pickup_city) = 1 THEN 'All Regions'
        ELSE pickup_city
    END AS region,
    COUNT(*) AS total_trips,
    SUM(
        CASE
            WHEN cancellation_reason = 'Rider no show' THEN 1
            ELSE 0
        END
    ) AS rider_no_shows
FROM trips
GROUP BY ROLLUP(pickup_city)
ORDER BY GROUPING(pickup_city), pickup_city;
```

Database > postgres@localhost > q4_2.sql

Output | Result 14

CSV

region	total_trips	rider_no_shows
Danang	43	1
Haiphong	39	3
Hanoi	38	3
Hue	40	4
Saigon	40	1
All Regions	200	12

```sql
-- Part C: A two-step pipeline
-- Leadership wants to establish a baseline for evaluating high-volume producers. That needs two steps: use a WITH clause to create an intermediate result that summarizes performance for each producer, then

-- ✓  A WITH clause producing an intermediate aggregate.

-- ✓  A second aggregation or filter applied to that intermediate result.

WITH driver_performance AS (  341 ms
    SELECT
        driver_id,
        COUNT(*) AS total_trips,
        SUM(
            CASE
                WHEN cancellation_reason = 'Rider no show' THEN 1
                ELSE 0
            END
        ) AS rider_no_shows
    FROM trips
    GROUP BY driver_id
)
SELECT
    COUNT(*) AS high_volume_drivers,
    SUM(total_trips) AS total_trips,
    SUM(rider_no_shows) AS total_no_shows,
    ROUND(100.0 * SUM(rider_no_shows) / SUM(total_trips),2) AS failure_rate_percent
FROM driver_performance
WHERE total_trips >= 10;
```

Database > postgres@localhost > q4_2.sql

Output | Result 15

CSV

high_volume_drivers	total_trips	total_no_shows	failure_rate_percent
12	194	11	5.67